

## L2: Lab Report-OpenMP

### 1 Experiment Setup

Goal: describe the hardware and software configurations of the lab server, and fill out the following table

Hint: use Linux command

| Configuration          | Configuration instructions |                                                                                |
|------------------------|----------------------------|--------------------------------------------------------------------------------|
| Hardware configuration | CPU                        | Intel(R) Xeon(R) Gold 6330<br>CPU @ 2.00GHz                                    |
|                        | Memory                     | 256 GiB                                                                        |
|                        | Storage                    | 1.8 TB local disk (47 GB available), 57 TB network storage (Lustre filesystem) |
| Software configuration | Operating System           | Ubuntu 20.04 (Linux kernel 5.4.0-208-generic)                                  |
|                        | Compiler                   | g++ 9.4.0                                                                      |
|                        | OpenMP version             | OpenMP 4.5                                                                     |

## 2. Performance Evaluation of K-means

### 2.1 Performance of Sequential K-means

Goal: measure the performance of sequential K-means with different input array size

Hint: 1) use the code template to generate the data points input arrays with

10<sup>4</sup>/10<sup>5</sup>/10<sup>6</sup> float-point elements and the cluster input arrays with 2/4/6/8 integer elements, 2) measure the execution time of the sequential code template for each input array, and 3) report the execution time for each run

#### Implementation Details:

The serial version initialize k cluster centers using the first k data points, then iterate: for each point compute the Euclidean distance to every center and assign it to the nearest one, accumulate the feature sums and counts per cluster, then update each center as the mean of its assigned points. Delta tracks the fraction of points that changed assignment. The loop terminates when  $\text{delta} \leq \text{threshold}$  or 1000 iterations are reached.

## Pseudocode:

```
FUNCTION serial_clustering(feature, nfeatures, npoints, nclusters, threshold):

    // Initialize cluster centers
    clusters[0..nclusters-1] <- first nclusters data points

    // Initialize membership and accumulators
    membership[i] <- -1 for all i
    allocate new_centers[nclusters][nfeatures]
    allocate new_centers_len[nclusters]

    DO:
        delta <- 0

        // Reset accumulators
        new_centers[k][j] <- 0 for all k, j
        new_centers_len[k] <- 0 for all k

        // --- ASSIGNMENT STEP ---
        FOR i = 0 TO npoints-1:
            index <- find_nearest_point(feature[i], clusters)
            IF membership[i] != index:
                delta <- delta + 1
                membership[i] <- index
                new_centers_len[index]++
                new_centers[index][j] += feature[i][j] for all j

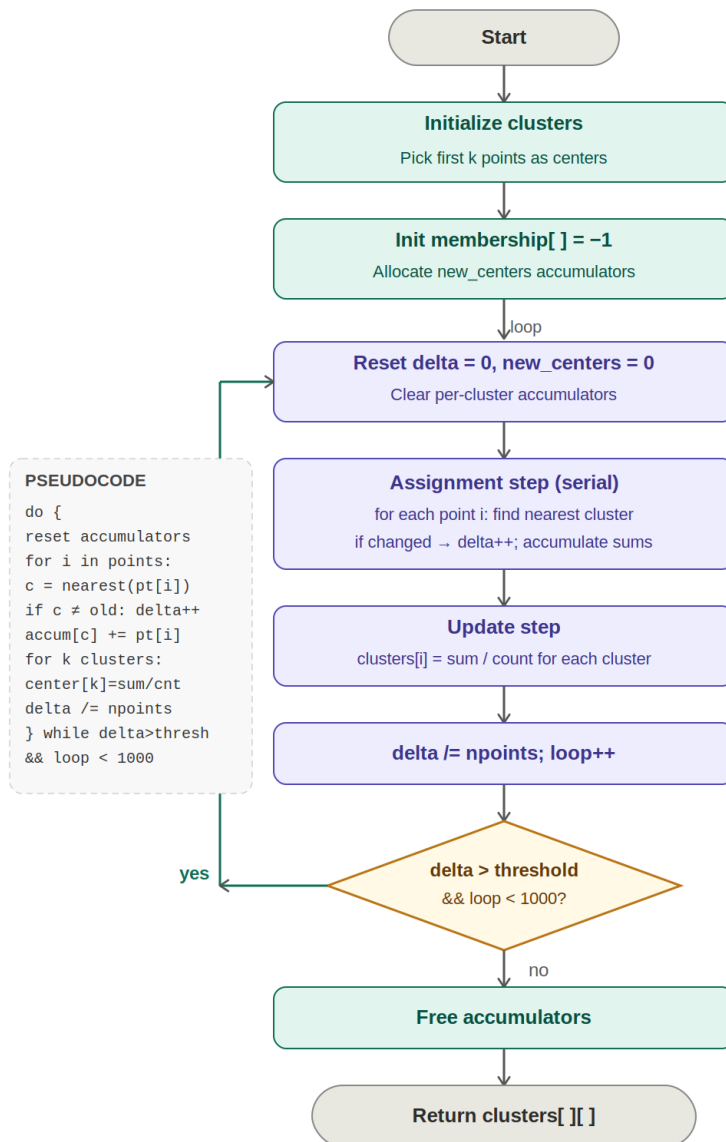
        // --- UPDATE STEP ---
        FOR k = 0 TO nclusters-1:
            IF new_centers_len[k] > 0:
                clusters[k][j] <- new_centers[k][j] / new_centers_len[k] for all j

        delta <- delta / npoints
        loop++

    WHILE delta > threshold AND loop < 1000

    free new_centers, new_centers_len
    RETURN clusters
```

## Flowchart:



Result:

| Input Arrays    | Cluster | Execution Time (s) |
|-----------------|---------|--------------------|
| 10 <sup>4</sup> | 2       | 0.016076           |
|                 | 4       | 0.084109           |
|                 | 6       | 0.110917           |
|                 | 8       | 0.109186           |
|                 | 16      | 0.167915           |
| 10 <sup>5</sup> | 2       | 0.167442           |
|                 | 4       | 0.881812           |
|                 | 6       | 0.916531           |
|                 | 8       | 4.188877           |
|                 | 16      | 6.774404           |
| 10 <sup>6</sup> | 2       | 1.545625           |
|                 | 4       | 6.653500           |
|                 | 6       | 13.074916          |
|                 | 8       | 18.419622          |
|                 | 16      | 45.460202          |

The performance results show that the execution time of the sequential K-means algorithm increases as both the dataset size and number of clusters increase. For 10<sup>4</sup> elements, execution times were below 0.2 s, while for 10<sup>5</sup> and 10<sup>6</sup> elements, runtimes increased significantly, reaching 6.77 s and 45.46 s respectively for 16 clusters. Overall, larger datasets and higher cluster counts require more distance computations, resulting in substantially higher execution times.

## 2.2 Performance of Parallel K-means

Goal: measure the performance speed up of parallel K-means with different input array size

Hint: 1) use the code template to generate the data points input arrays with  $10^4/10^5/10^6$  float-point elements and the cluster input arrays with 2/4/6/8 integer elements, 2) measure the execution time of the parallel code template for each input array, 3) increase the thread number from [1,2,4,8,16,32,64], and report the execution time for each run, 4) calculate the performance speedup compared to sequential code (speedup = sequential execution time / parallel execution time), 5) use line chart to report the scalability for each input array when increasing the thread number from [1,2,4,8,16,32,64]

Implementation Details:

How the problem is partitioned:

The assignment step is the bottleneck, each point's nearest-cluster search is independent of every other point. The implementation partition the npoints array evenly across T threads using `#pragma omp for schedule(static)`, so thread tid processes a contiguous chunk of npoints / T rows.

Parallel features used:

- `#pragma omp parallel` with a `reduction(+:delta)` clause — each thread accumulates its own delta and OpenMP merges them automatically at the barrier.
- `#pragma omp for schedule(static)` for the assignment loop — static gives even load since all points cost roughly the same.
- Per-thread accumulators `partial_new_centers[tid][k][f]` and `partial_new_centers_len[tid][k]`, each thread writes only to its own row, eliminating all race conditions on the accumulate step without any locks.

### Optimizations:

- False-sharing avoidance — by allocating `partial_new_centers[T][k][f]` as a 3D array where the thread index is outermost, different threads write to different cache lines.
- Manual reduction instead of atomic — combining partials with a sequential double loop after the parallel region is faster than `#pragma omp atomic` inside the hot loop, since it eliminates repeated lock contention.
- The update step (dividing sums by counts) and the reduction loop are both sequential but cheap compared to the  $O(n \times k)$  assignment step, so they don't hurt scalability.

## Pseudocode:

```
FUNCTION parallel_clustering(feature, nfeatures, npoints, nclusters, threshold, T):

    // Initialize cluster centers
    clusters[0..nclusters-1] <- first nclusters data points

    // Initialize membership
    membership[i] <- -1 for all i

    // Allocate global accumulators
    allocate new_centers[nclusters][nfeatures]
    allocate new_centers_len[nclusters]

    // Allocate thread-local accumulators (one per thread)
    allocate partial_new_centers[T][nclusters][nfeatures]
    allocate partial_new_centers_len[T][nclusters]

    DO:
        delta <- 0

        // Reset global accumulators
        new_centers[k][j] <- 0 for all k, j
        new_centers_len[k] <- 0 for all k

        // Reset thread-local accumulators
        partial_new_centers[t][k][j] <- 0 for all t, k, j
        partial_new_centers_len[t][k] <- 0 for all t, k

        // --- PARALLEL REGION ---
        #pragma omp parallel reduction(+:delta)

        tid <- omp_get_thread_num()

        #pragma omp for schedule(static) // divide npoints evenly across threads
        FOR i = 0 TO npoints-1:
            index <- find_nearest_point(feature[i], clusters)
            IF membership[i] != index:
                delta <- delta + 1.0 // safe: OMP reduction handles merge
                membership[i] <- index
                partial_new_centers_len[tid][index]++
                partial_new_centers[tid][index][j] += feature[i][j] for all j

        // --- END PARALLEL (implicit barrier here) ---
```



```

// Manual reduction: merge thread-local into global (main thread)
FOR k = 0 TO nclusters-1:
  FOR t = 0 TO T-1:
    new_centers_len[k]    += partial_new_centers_len[t][k]
    new_centers[k][j]    += partial_new_centers[t][k][j]  for all j

// --- UPDATE STEP ---
FOR k = 0 TO nclusters-1:
  IF new_centers_len[k] > 0:
    clusters[k][j] <- new_centers[k][j] / new_centers_len[k]  for all j

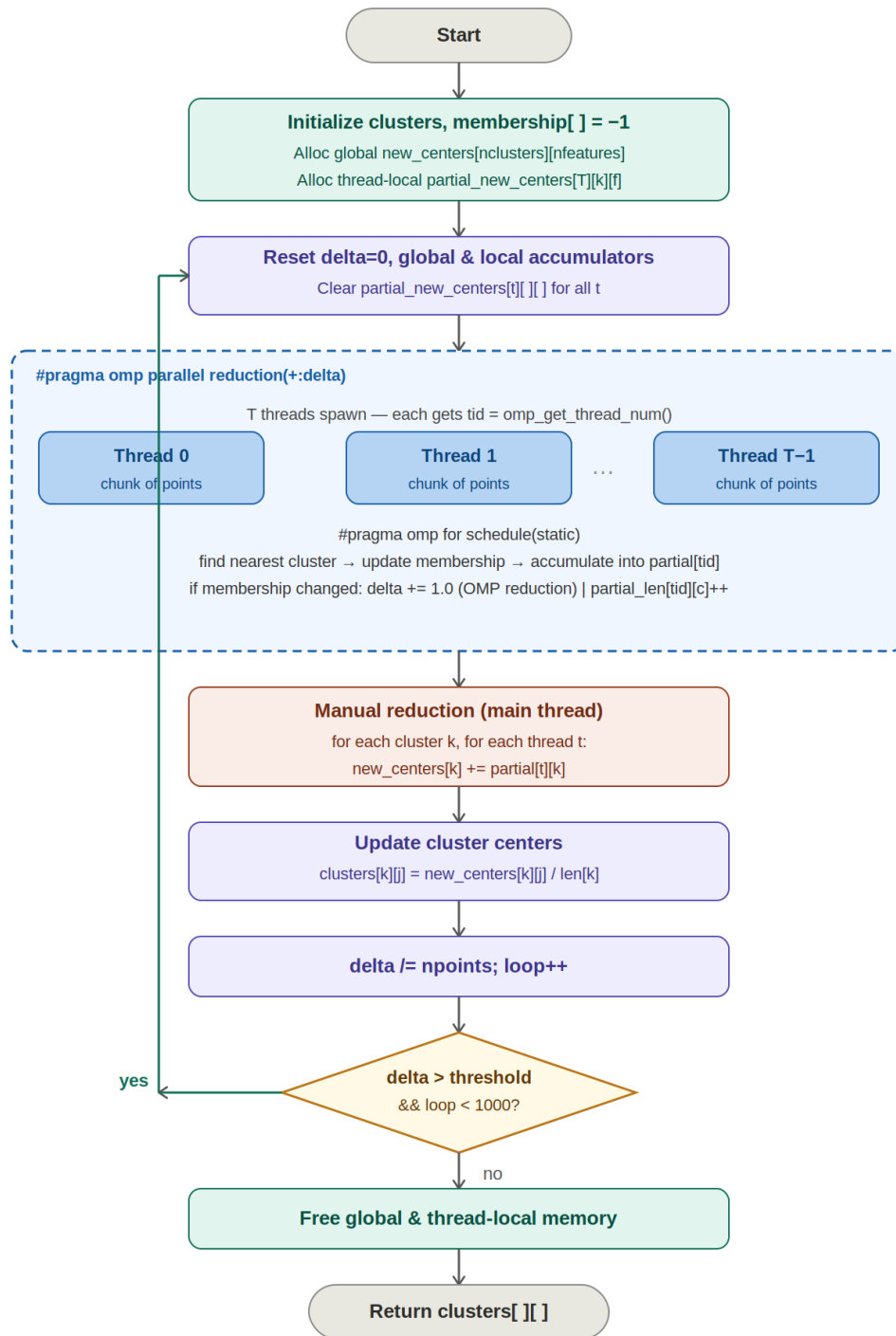
delta <- delta / npoints
loop++

WHILE delta > threshold AND loop < 1000

free all memory
RETURN clusters

```

Flowchart:



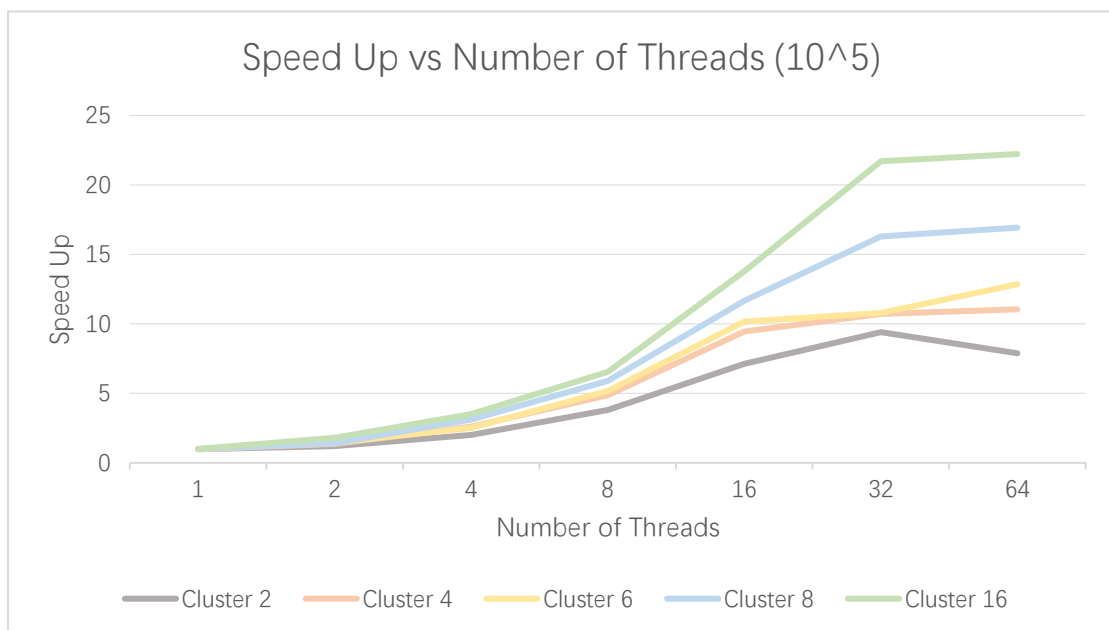
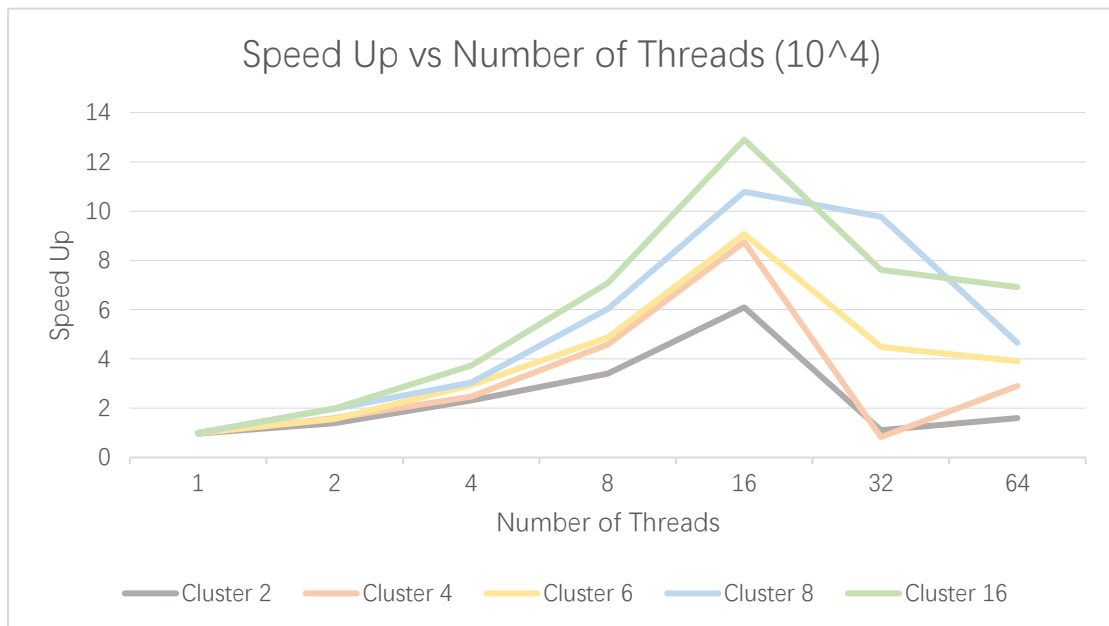
Results:

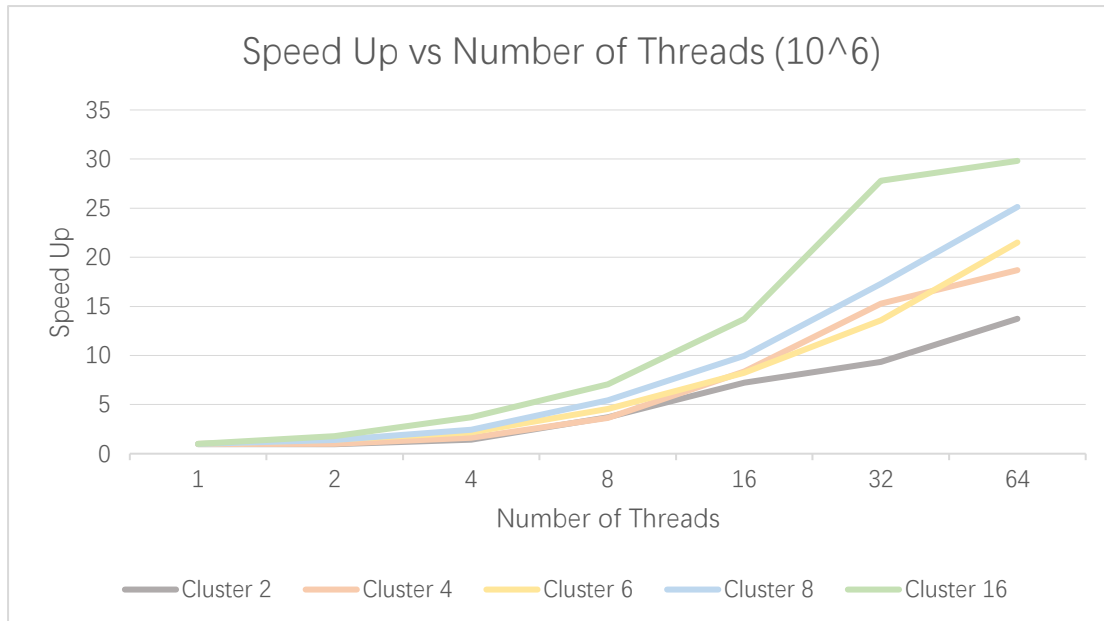
| Input Arrays    | Cluster | Threads | Execution Time (s) | Speed Up    |
|-----------------|---------|---------|--------------------|-------------|
| 10 <sup>4</sup> | 2       | 1       | 0.016524           | 0.972887921 |
|                 |         | 2       | 0.011491           | 1.399007919 |
|                 |         | 4       | 0.006937           | 2.317428283 |
|                 |         | 8       | 0.004719           | 3.406653952 |
|                 |         | 16      | 0.002638           | 6.094010614 |
|                 |         | 32      | 0.014536           | 1.105943864 |
|                 |         | 64      | 0.010050           | 1.59960199  |
|                 | 4       | 1       | 0.086181           | 0.975957578 |
|                 |         | 2       | 0.052400           | 1.605133588 |
|                 |         | 4       | 0.033975           | 2.475614422 |
|                 |         | 8       | 0.018304           | 4.595115822 |
|                 |         | 16      | 0.009618           | 8.744957372 |
|                 |         | 32      | 0.100849           | 0.834009261 |
|                 |         | 64      | 0.029044           | 2.89591654  |
|                 | 6       | 1       | 0.111129           | 0.998092307 |
|                 |         | 2       | 0.070713           | 1.568551751 |
|                 |         | 4       | 0.037788           | 2.935243993 |
|                 |         | 8       | 0.022733           | 4.879118462 |
|                 |         | 16      | 0.012226           | 9.072223131 |
|                 |         | 32      | 0.024701           | 4.490385005 |
|                 |         | 64      | 0.028353           | 3.912002257 |
|                 | 8       | 1       | 0.109722           | 0.995114927 |
|                 |         | 2       | 0.055253           | 1.976109894 |
|                 |         | 4       | 0.035802           | 3.049717893 |
|                 |         | 8       | 0.018111           | 6.028711833 |
|                 |         | 16      | 0.010124           | 10.78486764 |
|                 |         | 32      | 0.011168           | 9.776683381 |
|                 |         | 64      | 0.023486           | 4.648982372 |
|                 | 16      | 1       | 0.167500           | 1.002477612 |
|                 |         | 2       | 0.084164           | 1.995092914 |
|                 |         | 4       | 0.045036           | 3.728461675 |
|                 |         | 8       | 0.023692           | 7.087413473 |
|                 |         | 16      | 0.013009           | 12.90760243 |
|                 |         | 32      | 0.022037           | 7.619685075 |
|                 |         | 64      | 0.024260           | 6.92147568  |

|                 |    |    |          |             |
|-----------------|----|----|----------|-------------|
| 10 <sup>5</sup> | 2  | 1  | 0.170316 | 0.983125484 |
|                 |    | 2  | 0.139047 | 1.204211526 |
|                 |    | 4  | 0.083300 | 2.010108043 |
|                 |    | 8  | 0.044124 | 3.794805548 |
|                 |    | 16 | 0.023496 | 7.126404494 |
|                 |    | 32 | 0.017806 | 9.403684151 |
|                 |    | 64 | 0.021239 | 7.883704506 |
|                 | 4  | 1  | 0.901435 | 0.978231376 |
|                 |    | 2  | 0.645110 | 1.36691727  |
|                 |    | 4  | 0.338876 | 2.602167164 |
|                 |    | 8  | 0.181093 | 4.869387552 |
|                 |    | 16 | 0.093242 | 9.457240299 |
|                 |    | 32 | 0.082287 | 10.71629784 |
|                 |    | 64 | 0.079822 | 11.04723009 |
|                 | 6  | 1  | 0.917432 | 0.999017911 |
|                 |    | 2  | 0.628154 | 1.459086466 |
|                 |    | 4  | 0.365131 | 2.510142935 |
|                 |    | 8  | 0.177347 | 5.168009608 |
|                 |    | 16 | 0.090266 | 10.15366805 |
|                 |    | 32 | 0.085077 | 10.77295861 |
|                 |    | 64 | 0.071333 | 12.84862546 |
|                 | 8  | 1  | 4.207732 | 0.995518964 |
|                 |    | 2  | 2.956271 | 1.416946214 |
|                 |    | 4  | 1.340278 | 3.125379212 |
|                 |    | 8  | 0.710938 | 5.892042625 |
|                 |    | 16 | 0.359018 | 11.66759605 |
|                 |    | 32 | 0.257351 | 16.27690197 |
|                 |    | 64 | 0.247511 | 16.92400338 |
|                 | 16 | 1  | 6.743815 | 1.00453586  |
|                 |    | 2  | 3.745796 | 1.808535222 |
|                 |    | 4  | 1.929457 | 3.511041708 |
|                 |    | 8  | 1.034102 | 6.551001739 |
|                 |    | 16 | 0.491967 | 13.77003742 |
|                 |    | 32 | 0.312238 | 21.69628296 |
|                 |    | 64 | 0.304912 | 22.21757097 |
| 10 <sup>6</sup> | 2  | 1  | 1.571312 | 0.983652515 |
|                 |    | 2  | 1.597728 | 0.967389318 |
|                 |    | 4  | 1.074890 | 1.437937835 |
|                 |    | 8  | 0.417301 | 3.703861242 |
|                 |    | 16 | 0.214170 | 7.216813746 |
|                 |    | 32 | 0.165139 | 9.359539539 |
|                 |    | 64 | 0.112517 | 13.7368131  |

|  |    |    |           |             |
|--|----|----|-----------|-------------|
|  | 4  | 1  | 6.782900  | 0.980922614 |
|  |    | 2  | 6.621827  | 1.004783121 |
|  |    | 4  | 4.073667  | 1.633295014 |
|  |    | 8  | 1.833244  | 3.629358667 |
|  |    | 16 | 0.794542  | 8.37400666  |
|  |    | 32 | 0.435554  | 15.27594741 |
|  |    | 64 | 0.356012  | 18.68897678 |
|  | 6  | 1  | 13.074254 | 1.000050634 |
|  |    | 2  | 9.160841  | 1.427261536 |
|  |    | 4  | 6.006685  | 2.17672743  |
|  |    | 8  | 2.860679  | 4.570563842 |
|  |    | 16 | 1.583610  | 8.256398987 |
|  |    | 32 | 0.963432  | 13.57118717 |
|  |    | 64 | 0.607948  | 21.50663544 |
|  | 8  | 1  | 18.493499 | 0.996005245 |
|  |    | 2  | 13.189331 | 1.396554685 |
|  |    | 4  | 7.518718  | 2.449835464 |
|  |    | 8  | 3.379216  | 5.450856648 |
|  |    | 16 | 1.847086  | 9.97226009  |
|  |    | 32 | 1.065235  | 17.2916042  |
|  |    | 64 | 0.733054  | 25.12723756 |
|  | 16 | 1  | 45.277452 | 1.004036225 |
|  |    | 2  | 25.357318 | 1.792784316 |
|  |    | 4  | 12.268859 | 3.705332501 |
|  |    | 8  | 6.429616  | 7.07043811  |
|  |    | 16 | 3.313718  | 13.71879019 |
|  |    | 32 | 1.636177  | 27.78440352 |
|  |    | 64 | 1.524720  | 29.81544284 |

## Graphs:





The parallel K-means performance results demonstrate that increasing the number of threads generally improves execution speed and reduces runtime. The speedup becomes more significant for larger datasets and higher cluster counts because more computations can be distributed across multiple threads.

For the ( $10^4$ ) dataset, the parallel implementation achieved moderate improvements. The best performance was usually observed around 16 threads, where the maximum speedup reached approximately  $12.9\times$  for 16 clusters. However, performance dropped when using 32 and 64 threads for some cluster configurations. This occurs because the dataset is relatively small, so the overhead of thread creation, synchronization, and communication becomes larger than the computational benefit of additional threads.

For the ( $10^5$ ) dataset, the scalability improved considerably. Increasing the thread count continuously reduced execution time, especially for larger cluster counts. The highest speedup reached about  $22.2\times$  for 16 clusters using 64 threads. Unlike the smaller dataset, the larger workload provided enough computations to efficiently utilize more processing cores, reducing the impact of parallel overhead.

The ( $10^6$ ) dataset produced the best parallel performance and scalability. Execution

time decreased consistently as the number of threads increased. For example, with 16 clusters, the runtime reduced from 45.28 s using one thread to only 1.52 s using 64 threads, achieving nearly  $29.8\times$  speedup. Similarly, configurations with 8 clusters and 64 threads achieved over  $25\times$  speedup. These results show that the parallel implementation becomes highly efficient for computationally intensive workloads.

The graphs also show that higher cluster counts generally produce larger speedups. This is because increasing the number of clusters increases the amount of distance calculations performed during the assignment step, which is the most computationally expensive part of K-means. Since these calculations are independent for each data point, they parallelize very effectively across multiple threads.

However, the results also indicate that parallel efficiency is not perfectly linear. In some cases, doubling the number of threads does not double the speedup. This is due to several factors such as thread scheduling overhead, synchronization costs, memory bandwidth limitations, and load imbalance between threads. These effects are especially visible for smaller datasets where computational work per thread is insufficient to fully utilize all available cores.

Overall, the parallel K-means implementation significantly outperforms the sequential version, particularly for large datasets and large cluster counts. The results confirm that parallelization is highly effective for K-means clustering because the assignment computations can be distributed independently across multiple threads, greatly reducing total execution time.



### 3. Performance Evaluation of Ocean Simulation

#### 3.1 Performance of Sequential Ocean Simulation

##### 3.1.1 Implementation Details

Goal: describe your code implementation

Hint: use diagram or pseudo-code

The red-black scheme breaks each timestep into two phases so that every cell updated in a phase depends only on cells from the previous phase (no RAW hazard within a phase). In the red phase, row  $i$  starts at column offset  $(i+1) \% 2$ , meaning alternating cells are updated. In the black phase the offset is  $i \% 2$ , covering the complementary set. Each cell is replaced by the 5-point average of itself and its four neighbors.

## Pseudocode:

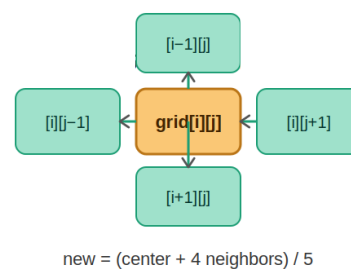
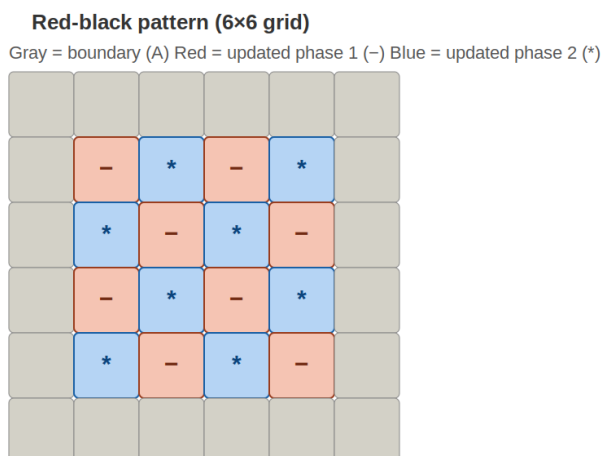
```
FUNCTION ocean(grid, dim, timesteps):

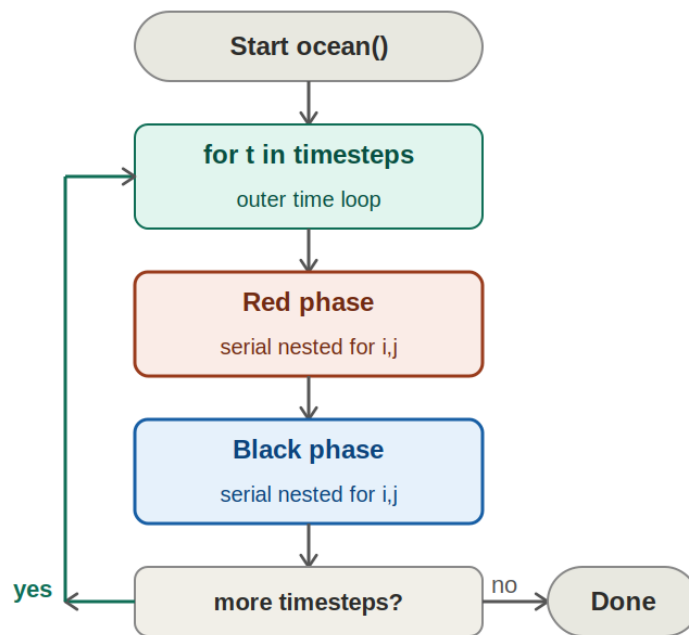
  FOR t = 0 TO timesteps-1:

    // --- RED PHASE ---
    // Updates cells where (i + j) is even (interior only)
    FOR i = 1 TO dim-2:
      offset <- (i + 1) % 2          // alternating column start per row
      FOR j = 1+offset TO dim-2 STEP 2:
        grid[i][j] <- (grid[i][j] +
                        grid[i][j-1] +
                        grid[i][j+1] +
                        grid[i-1][j] +
                        grid[i+1][j]) / 5

    // --- BLACK PHASE ---
    // Updates the complementary set of interior cells
    FOR i = 1 TO dim-2:
      offset <- i % 2              // complementary column start
      FOR j = 1+offset TO dim-2 STEP 2:
        grid[i][j] <- (grid[i][j] +
                        grid[i][j-1] +
                        grid[i][j+1] +
                        grid[i-1][j] +
                        grid[i+1][j]) / 5
```

## Diagram and Flowchart:





### 3.1.2 Performance Results

Goal: measure the performance of sequential Ocean Simulation with different input array size

Hint: 1) pass the argument of the dimension of the grid and the argument of time steps to the compiled Ocean Simulation program, 2) measure the execution time of the sequential code template for each pair of arguments, and 3) report the execution time for each run

Results:

| Dimension | Timesteps | Execution Time (ns) |
|-----------|-----------|---------------------|
| 18        | 4         | 1507                |
|           | 16        | 5541                |
|           | 64        | 21601               |
|           | 256       | 85847               |

|      |     |            |
|------|-----|------------|
| 66   | 4   | 20216      |
|      | 16  | 80533      |
|      | 64  | 321802     |
|      | 256 | 1292369    |
| 258  | 4   | 317393     |
|      | 16  | 1268193    |
|      | 64  | 5075213    |
|      | 256 | 20367930   |
| 1026 | 4   | 5522250    |
|      | 16  | 21821780   |
|      | 64  | 87630770   |
|      | 256 | 351664390  |
| 4098 | 4   | 93154781   |
|      | 16  | 367420295  |
|      | 64  | 1474409981 |
|      | 256 | 5863692773 |

The sequential Ocean Simulation results show that execution time increases steadily as both the grid dimension and the number of timesteps increase. For each grid size, the runtime grows almost linearly with the number of timesteps because the same computations are repeated at every simulation step.

In addition, larger grid dimensions produce a much greater increase in execution time since the total number of cells in the simulation grows quadratically ( $N \times N$ ). For example, small dimensions such as 18 complete very quickly, while large dimensions such as 4098 require several billion nanoseconds to finish.

Overall, the results indicate that the sequential implementation has limited scalability. As the simulation size becomes larger, the computational cost increases significantly, leading to much longer execution times.

## 3.2 Performance of Parallel Ocean Simulation

### 3.2.1 Implementation Details

Goal: describe your code implementation

Hint: use diagram or pseudo-code

How the problem is partitioned:

Each row of interior cells is independent within a phase (a row's red cells only read from neighbors that are not updated in the same phase). The implementation partitions the outer  $i$  loop across threads: `#pragma omp parallel for` distributes rows to threads. The bottom diagram shows 4 threads each owning a contiguous stripe of rows. The implicit barrier at the end of the red-phase `parallel for` ensures all red updates finish before any thread starts the black phase, this is the critical correctness guarantee.

Parallel features used:

- `#pragma omp parallel for private(i,j) shared(grid,dim) schedule(dynamic)` applied once to the red phase and once to the black phase, inside the serial time loop.
- `schedule(dynamic)` — rows near the interior may have slightly varying work if the grid is non-square or irregular; dynamic scheduling rebalances automatically.

Optimizations:

- The time loop itself is kept serial — parallelizing across timesteps would require buffering two full grids and is not done here.
- `private(i,j)` prevents threads from sharing loop-counter variables.
- The two-phase split means zero synchronization within a phase — threads never write a cell that another thread in the same phase is reading, so no barriers or atomics are needed inside the loop body.

### Pseudocode:

```
FUNCTION ocean_parallel(grid, dim, timesteps, T):

    omp_set_num_threads(T)

    FOR t = 0 TO timesteps-1:          // outer time loop stays serial

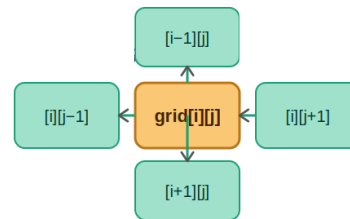
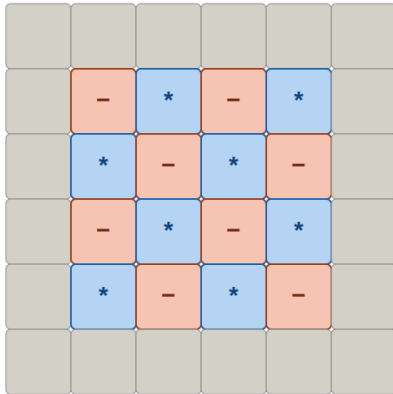
        // --- RED PHASE (parallelised over rows) ---
        #pragma omp parallel for private(i, j) shared(grid, dim) schedule(dynamic)
        FOR i = 1 TO dim-2:
            offset <- (i + 1) % 2
            FOR j = 1+offset TO dim-2 STEP 2:
                grid[i][j] <- (grid[i][j] +
                               grid[i][j-1] +
                               grid[i][j+1] +
                               grid[i-1][j] +
                               grid[i+1][j]) / 5
            // implicit barrier — all red updates finish before black phase begins

        // --- BLACK PHASE (parallelised over rows) ---
        #pragma omp parallel for private(i, j) shared(grid, dim) schedule(dynamic)
        FOR i = 1 TO dim-2:
            offset <- i % 2
            FOR j = 1+offset TO dim-2 STEP 2:
                grid[i][j] <- (grid[i][j] +
                               grid[i][j-1] +
                               grid[i][j+1] +
                               grid[i-1][j] +
                               grid[i+1][j]) / 5
            // implicit barrier
```

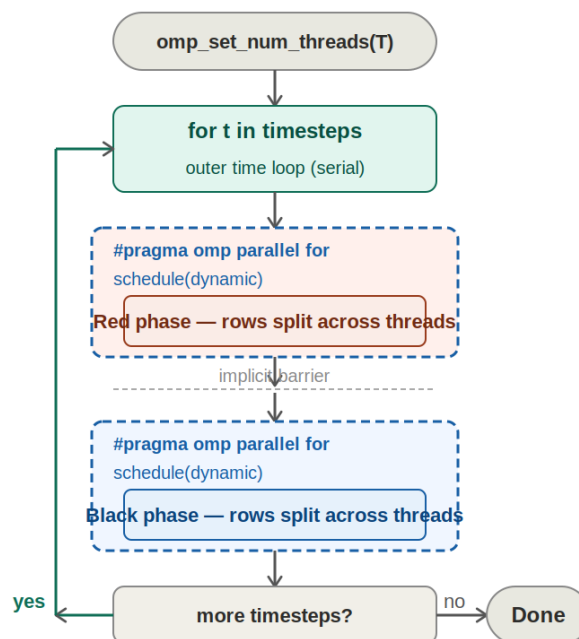
### Diagram and Flowchart:

### Red-black pattern (6×6 grid)

Gray = boundary (A) Red = updated phase 1 (–) Blue = updated phase 2 (\*)



$$\text{new} = (\text{center} + 4 \text{ neighbors}) / 5$$



### Row partitioning across threads (per phase)



## 3.2.2 Performance Results

Goal: measure the performance speed up of parallel Ocean Simulation with different input array size

Hint: 1) pass the argument of the dimension of the grid and the argument of time steps to the compiled Ocean Simulation program, 2) measure the execution time of the parallel code template for each pair of arguments, 3) increase the thread number from [1,2,4,8,16,32,64], and report the execution time for each run, 4) calculate the performance speedup compared to sequential code (speedup = sequential execution time / parallel execution time), 5) use line chart to report the scalability for each input array when increasing the thread number from [1,2,4,8,16,32,64]

Result:

| Dimension | Timesteps | Threads | Execution Time (ns) | Speed Up    |
|-----------|-----------|---------|---------------------|-------------|
| 18        | 4         | 1       | 14840               | 0.101549865 |
|           |           | 2       | 50707               | 0.029719763 |
|           |           | 4       | 82213               | 0.018330434 |
|           |           | 8       | 136417              | 0.01104701  |
|           |           | 16      | 268644              | 0.005609654 |
|           |           | 32      | 12323781            | 0.000122284 |
|           |           | 64      | 1631135             | 0.000923897 |
|           | 16        | 1       | 32737               | 0.169258026 |
|           |           | 2       | 100000              | 0.05541     |
|           |           | 4       | 143056              | 0.038733084 |
|           |           | 8       | 209442              | 0.026456012 |
|           |           | 16      | 338340              | 0.016377017 |
|           |           | 32      | 14793996            | 0.000374544 |
|           |           | 64      | 4054227             | 0.001366722 |
|           | 64        | 1       | 95794               | 0.22549429  |
|           |           | 2       | 329566              | 0.065543776 |
|           |           | 4       | 418300              | 0.051639971 |
|           |           | 8       | 505268              | 0.042751569 |
|           |           | 16      | 726127              | 0.02974824  |
|           |           | 32      | 27956890            | 0.000772654 |
|           |           | 64      | 13229590            | 0.001632779 |
|           | 256       | 1       | 349886              | 0.245357059 |
|           |           | 2       | 1231807             | 0.069691924 |
|           |           | 4       | 1513362             | 0.056726018 |
|           |           | 8       | 1681626             | 0.051049996 |



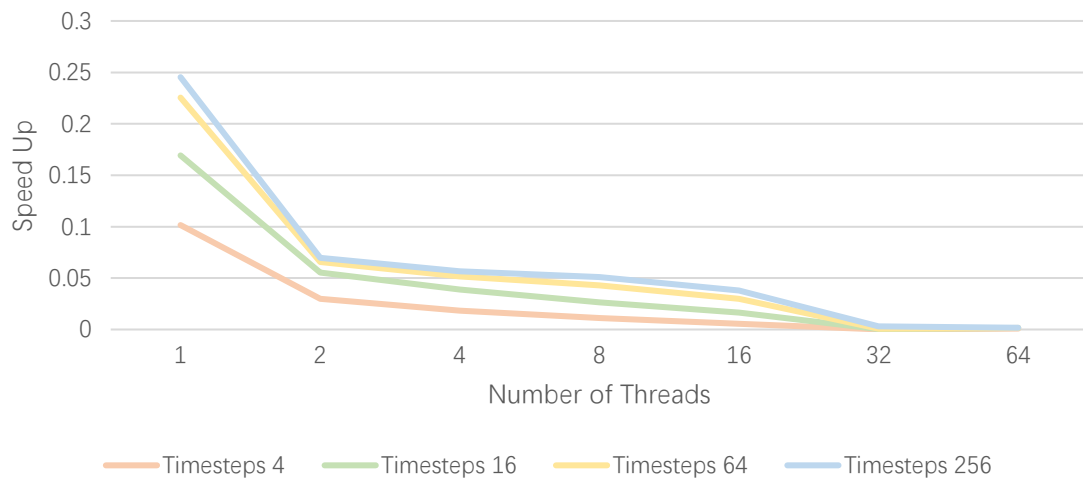
|     |     |    |          |             |
|-----|-----|----|----------|-------------|
|     |     | 16 | 2259257  | 0.03799789  |
|     |     | 32 | 28780309 | 0.002982838 |
|     |     | 64 | 50166433 | 0.001711244 |
| 66  | 4   | 1  | 35021    | 0.577253648 |
|     |     | 2  | 144667   | 0.139741613 |
|     |     | 4  | 154978   | 0.130444321 |
|     |     | 8  | 194515   | 0.103930288 |
|     |     | 16 | 314570   | 0.064265505 |
|     |     | 32 | 21514225 | 0.000939657 |
|     |     | 64 | 1608344  | 0.01256945  |
|     | 16  | 1  | 126803   | 0.63510327  |
|     |     | 2  | 508866   | 0.158259738 |
|     |     | 4  | 459206   | 0.175374451 |
|     |     | 8  | 462504   | 0.174123899 |
|     |     | 16 | 502330   | 0.160318914 |
|     |     | 32 | 51293583 | 0.00157004  |
|     |     | 64 | 4083096  | 0.019723514 |
|     | 64  | 1  | 473081   | 0.680226008 |
|     |     | 2  | 1944879  | 0.165461193 |
|     |     | 4  | 1697800  | 0.189540582 |
|     |     | 8  | 1530774  | 0.210221757 |
|     |     | 16 | 1438362  | 0.223728102 |
|     |     | 32 | 12726808 | 0.025285366 |
|     |     | 64 | 13303574 | 0.024189139 |
|     | 256 | 1  | 1846890  | 0.699754181 |
|     |     | 2  | 7920136  | 0.163175102 |
|     |     | 4  | 6868780  | 0.188151171 |
|     |     | 8  | 5839103  | 0.221330057 |
|     |     | 16 | 5031512  | 0.256854997 |
|     |     | 32 | 64271322 | 0.02010802  |
|     |     | 64 | 51186862 | 0.025248061 |
| 258 | 4   | 1  | 351440   | 0.903121443 |
|     |     | 2  | 937095   | 0.338698851 |
|     |     | 4  | 816920   | 0.388523968 |
|     |     | 8  | 626382   | 0.506708366 |
|     |     | 16 | 564054   | 0.562699671 |
|     |     | 32 | 12548345 | 0.025293614 |
|     |     | 64 | 1721023  | 0.184421126 |
|     | 16  | 1  | 1385968  | 0.915023291 |
|     |     | 2  | 3788001  | 0.33479215  |
|     |     | 4  | 3110383  | 0.407728887 |
|     |     | 8  | 2238070  | 0.566645815 |

|      |     |    |           |             |
|------|-----|----|-----------|-------------|
|      |     | 16 | 1648073   | 0.769500501 |
|      |     | 32 | 23489623  | 0.0539895   |
|      |     | 64 | 4303797   | 0.294668406 |
|      | 64  | 1  | 5503822   | 0.922125207 |
|      |     | 2  | 15309426  | 0.331509032 |
|      |     | 4  | 12362861  | 0.410520914 |
|      |     | 8  | 8628612   | 0.588184171 |
|      |     | 16 | 6090653   | 0.83327896  |
|      |     | 32 | 29481864  | 0.172146951 |
|      |     | 64 | 14530851  | 0.34927156  |
|      | 256 | 1  | 22055303  | 0.923493547 |
|      |     | 2  | 60963472  | 0.334100558 |
|      |     | 4  | 49733721  | 0.409539636 |
|      |     | 8  | 34504941  | 0.590290243 |
|      |     | 16 | 23853315  | 0.853882574 |
|      |     | 32 | 27871550  | 0.730778518 |
|      |     | 64 | 54691519  | 0.37241478  |
| 1026 | 4   | 1  | 5628801   | 0.981070391 |
|      |     | 2  | 4824772   | 1.144561857 |
|      |     | 4  | 4349955   | 1.269495891 |
|      |     | 8  | 2921943   | 1.889923931 |
|      |     | 16 | 1893189   | 2.916903701 |
|      |     | 32 | 20898641  | 0.26423967  |
|      |     | 64 | 2421442   | 2.280562574 |
|      | 16  | 1  | 22449723  | 0.97202892  |
|      |     | 2  | 19390145  | 1.125405715 |
|      |     | 4  | 17508841  | 1.246329212 |
|      |     | 8  | 11389916  | 1.915885947 |
|      |     | 16 | 6877633   | 3.172861942 |
|      |     | 32 | 42441292  | 0.514163895 |
|      |     | 64 | 7322424   | 2.980130623 |
|      | 64  | 1  | 89996376  | 0.97371443  |
|      |     | 2  | 80179705  | 1.092929564 |
|      |     | 4  | 70303752  | 1.246459364 |
|      |     | 8  | 45560958  | 1.923374175 |
|      |     | 16 | 26833230  | 3.265755558 |
|      |     | 32 | 41545308  | 2.109281992 |
|      |     | 64 | 26968282  | 3.249401278 |
|      | 256 | 1  | 355894898 | 0.988113041 |
|      |     | 2  | 325241872 | 1.081239595 |
|      |     | 4  | 307066304 | 1.145239271 |
|      |     | 8  | 180235644 | 1.951136757 |

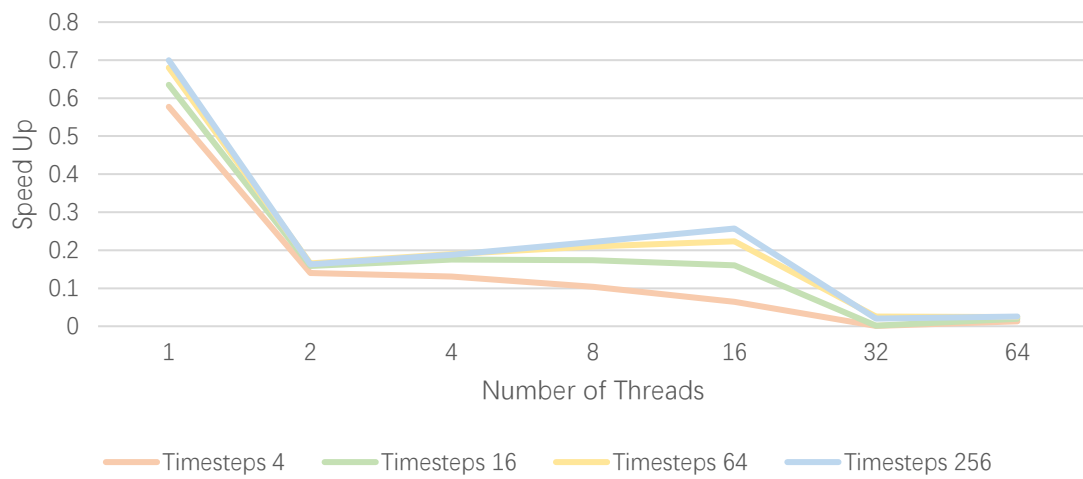
|      |     |    |             |             |
|------|-----|----|-------------|-------------|
| 4098 |     | 16 | 105879680   | 3.32135864  |
|      |     | 32 | 90307526    | 3.894076226 |
|      |     | 64 | 104784393   | 3.356076033 |
|      | 4   | 1  | 93561933    | 0.995648316 |
|      |     | 2  | 67330262    | 1.383549956 |
|      |     | 4  | 44158075    | 2.109575225 |
|      |     | 8  | 26648842    | 3.495640861 |
|      |     | 16 | 18780476    | 4.960192755 |
|      |     | 32 | 16126681    | 5.7764385   |
|      |     | 64 | 12854273    | 7.246989464 |
|      | 16  | 1  | 371896078   | 0.987964963 |
|      |     | 2  | 340282694   | 1.079750165 |
|      |     | 4  | 175865105   | 2.08921659  |
|      |     | 8  | 105586543   | 3.479802298 |
|      |     | 16 | 60790812    | 6.044010319 |
|      |     | 32 | 51729848    | 7.102674939 |
|      |     | 64 | 47704325    | 7.702033201 |
|      | 64  | 1  | 1481812621  | 0.995004335 |
|      |     | 2  | 1953530325  | 0.754741282 |
|      |     | 4  | 1194222740  | 1.234618913 |
|      |     | 8  | 488668882   | 3.017196378 |
|      |     | 16 | 247687224   | 5.952709055 |
|      |     | 32 | 202461717   | 7.282413697 |
|      |     | 64 | 238525466   | 6.181352481 |
|      | 256 | 1  | 5944182526  | 0.986459071 |
|      |     | 2  | 11053869200 | 0.530465185 |
|      |     | 4  | 5604961854  | 1.046161049 |
|      |     | 8  | 2851259260  | 2.056527393 |
|      |     | 16 | 1174973503  | 4.990489367 |
|      |     | 32 | 709799243   | 8.261058082 |
|      |     | 64 | 964754567   | 6.077911392 |

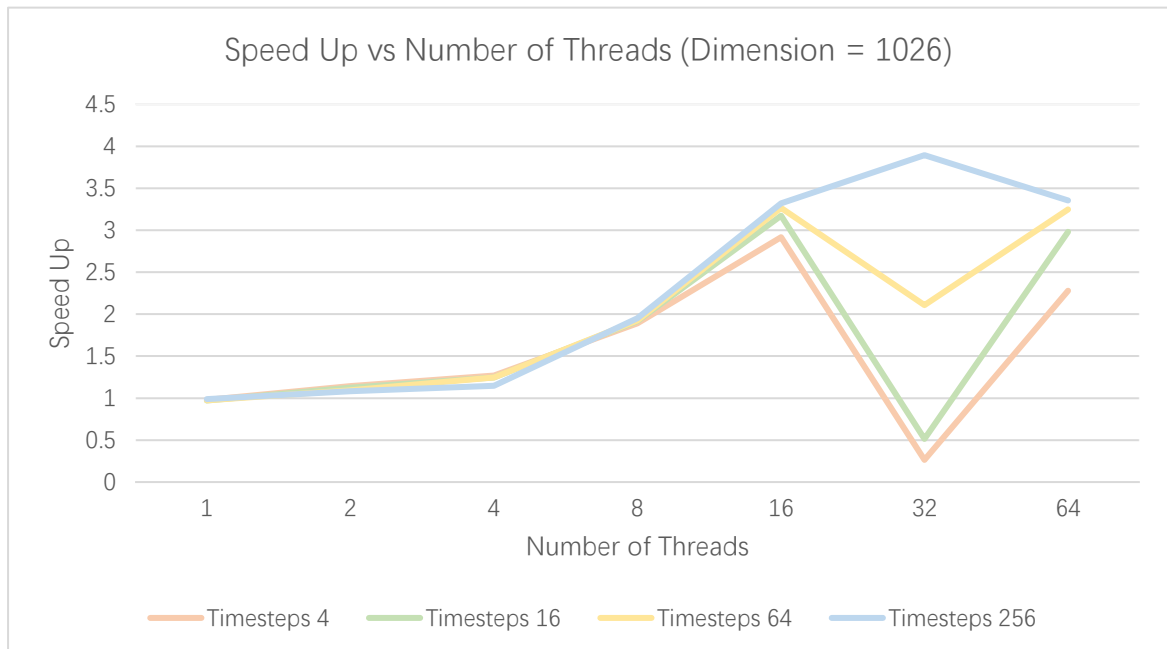
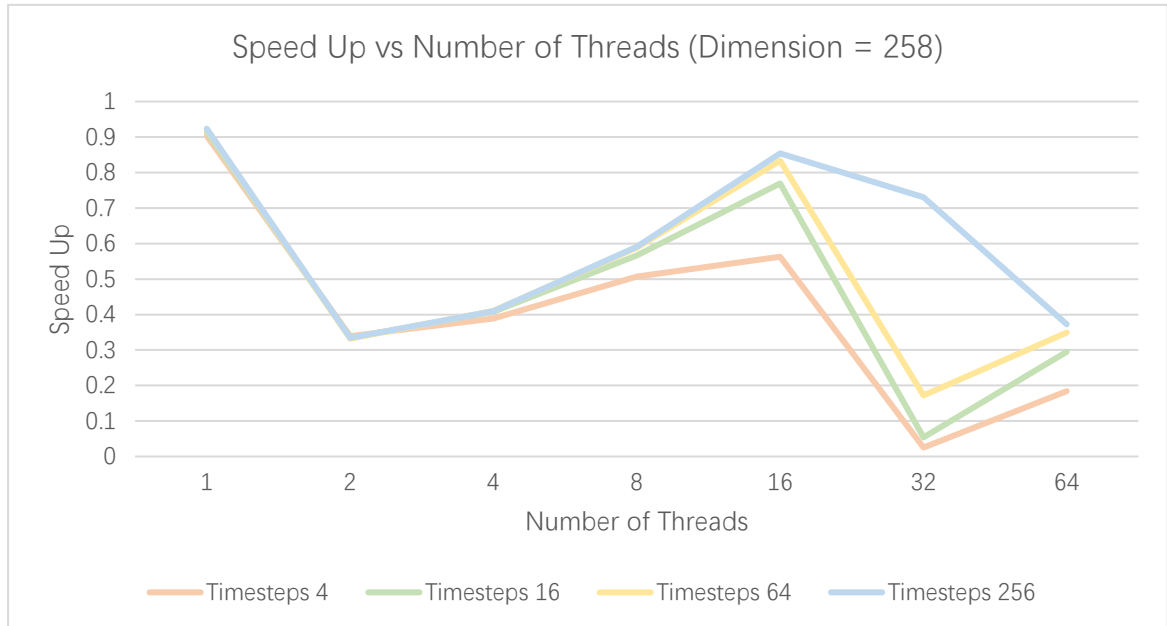
Graph:

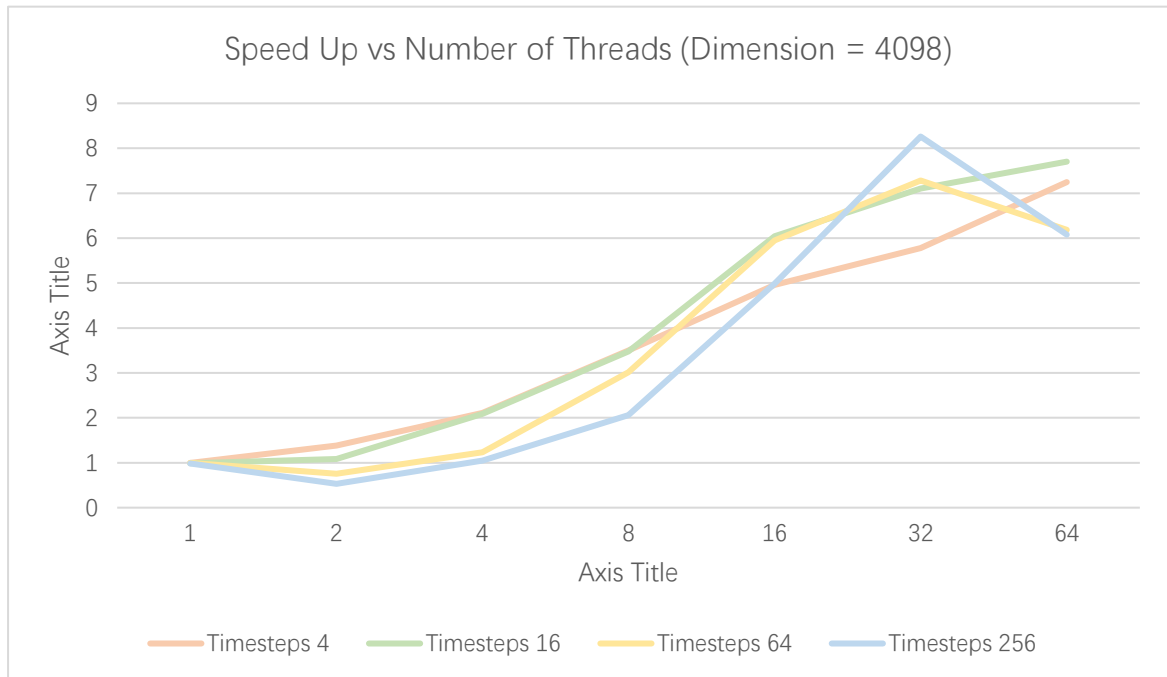
Speed Up vs Number of Threads (Dimension = 18)



Speed Up vs Number of Threads (Dimension = 66)







The parallel Ocean Simulation results show that performance depends heavily on the input size and the number of threads used.

For small grid dimensions such as 18, 66, and 258, the parallel version performs worse than the sequential version, with speedup values mostly below 1. This happens because the overhead of thread creation, synchronization, and communication is larger than the actual computation work. As a result, adding more threads increases execution time instead of improving performance.

For medium and large grid dimensions such as 1026 and 4098, parallelization becomes effective. The computation workload is large enough to utilize multiple threads efficiently, so the speedup increases as the thread count grows. For example, dimension 4098 with 256 timesteps achieves a maximum speedup of about 8.26 $\times$  using 32 threads. This demonstrates that parallel processing significantly reduces execution time for large simulations.

The results also show that performance improves up to an optimal thread count, usually around 16 or 32 threads. Beyond this point, speedup may decrease slightly, especially

at 64 threads, due to thread management overhead, memory contention, and limited hardware resources.

Overall, the results indicate that parallel Ocean Simulation scales well for large problem sizes but provides little or no benefit for small workloads. The effectiveness of parallelization increases as both the grid dimension and the number of timesteps grow.